

PROGRAMA INTEGRAL DE FORMACIÓN EN PYTHON

CURSO 1: FUNDAMENTOS BÁSICOS DE PYTHON (NIVEL 1
(PRINCIPIANTES))

Clase 01: El Panorama General de la Programación

«El Asistente, las Cajas, el Semáforo y la Licuadora»

Una inmersión panorámica e intuitiva a los 4 pilares fundamentales del pensamiento lógico de
un programador: Variables, Condicionales, Bucles y Funciones.

Instructor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Nivel del Curso: Principiante Absoluto | **Python:** 3.10+ | **Licencia:** MIT

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre **wisrovi SUITE** en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Estructura Pedagógica de Cada Guía

Cada documento de esta serie está diseñado rigurosamente bajo el estándar de ingeniería de software profesional:

- **Fundamento Conceptual y Metáfora Intuitiva:** Anclaje visual del concepto abstracto a la realidad.
- **Diagrama de Flujo y Arquitectura Mental:** Representación visual de cómo fluye la información.
- **Código de Nivel Profesional:** Sintaxis limpia, comentarios línea a línea y tipado estático (PEP 484).
- **Patrones y Gotchas:** Errores comunes que cometen los desarrolladores y cómo evitarlos.

Índice General de la Sesión

Sección	Contenido y Enfoque Temático	Página
Capítulo 1	Fundamentos y Metáfora Conceptual: Los Cuatro Pilares Fundamentales del Software	Pág. 4
Capítulo 2	Diagrama de Flujo y Arquitectura: Diagrama de Ejecución Secuencial y Control	Pág. 5
Capítulo 3	Implementación Práctica en Python: Script Integrador de los 4 Pilares	Pág. 6
Capítulo 4	Patrones Avanzados, Gotchas y Debugging: Buenas Prácticas, Gotchas y Depuración	Pág. 7
Cierre	Conclusiones, Notas del Mentor y Agradecimiento	Pág. 8
Anexos	Bibliografía Oficial y Enlaces de Profundización	Pág. 9

Objetivos de Aprendizaje (Competencias Clave)

Competencia Conceptual

Comprender que programar es dar instrucciones secuenciales precisas y dominar la función mental de los 4 pilares.

Competencia Práctica

Ejecutar tu primer script en VS Code usando `print()`, variables, condicionales `if` y funciones `def`.

Requisitos Previos y Entorno Recomendado

Para aprovechar al máximo esta sesión se recomienda tener instalado Python 3.10 o superior, Visual Studio Code con la extensión oficial de Python configurada, y haber completado las lecturas y ejercicios de las sesiones anteriores de la ruta formativa.

1. Los Cuatro Pilares Fundamentales del Software

Toda aplicación moderna, desde un script de automatización hasta una Inteligencia Artificial, está construida sobre cuatro bloques lógicos elementales.

☀ **Metáfora Central: El Asistente, las Cajas, el Semáforo y la Licuadora**

Imagina que la computadora es un asistente súper eficiente pero literal: las variables son cajas etiquetadas donde guarda cosas, el `if` es un semáforo que decide el camino según la luz, el `for` es una cinta transportadora que procesa elementos uno a uno, y la función `def` es una licuadora que recibe ingredientes y entrega un licuado.

Principios Teóricos y Modelo Mental

1. Variables (Memoria): Espacios con nombre para retener datos temporalmente. 2. Condicionales (Decisión): Bifurcaciones lógicas según condiciones booleanas. 3. Bucles (Repetición): Automatización de tareas repetitivas sin duplicar código. 4. Funciones (Modularidad): Bloques reutilizables con entradas y salidas bien definidas.

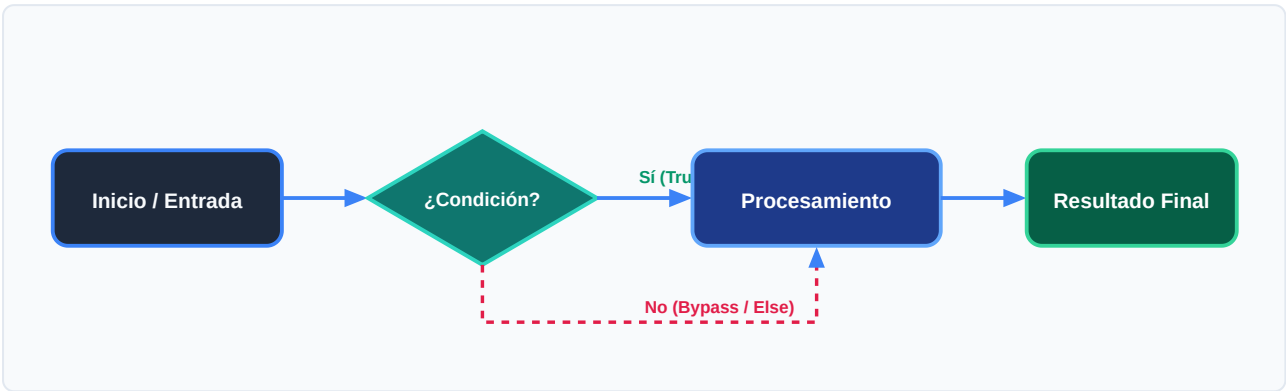
La magia del software no radica en la complejidad de cada pieza aislada, sino en la sinergia con la que se combinan para modelar la realidad.

⚡ **Regla de Oro en Python**

Python es un lenguaje interpretado, de tipado dinámico y fuertemente tipado: respeta la indentación y la semántica.

2. Diagrama de Ejecución Secuencial y Control

Cómo el intérprete de Python procesa el código línea por línea desde el punto de entrada hasta la resolución.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Lee la instrucción inicial e inicializa el entorno de variables en memoria.	Tabla de símbolos vacía -> asigna valores
2. Evaluación	Evalúa expresiones booleanas en condicionales para determinar la ruta.	Evalúa True o False en CPU
3. Transformación	Ejecuta el bloque indentado correspondiente a la condición satisfecha.	Transformación de variables
4. Retorno / Salida	Invoca funciones y devuelve el resultado a la consola con print().	Liberación de stack frame

Visualización Mental

Piensa en el intérprete de Python como un lector con un marcador que avanza de arriba a abajo, saltando sólo cuando encuentra estructuras de control.

3. Script Integrador de los 4 Pilares

Código autónomo que demuestra la interacción armónica entre variables, condicionales, bucles y funciones:

main.py (Python 3.10+)

UTF-8 • PEP 8 Compliant

```
# 1. Definición de Función Reutilizable (La Licuadora)
def evaluar_estudiante(nombre: str, nota: float) -> str:
    if nota >= 7.0:
        return f"¡Felicidades {nombre}! Aprobaste con éxito 🚀"
    else:
        return f"Ánimo {nombre}, debes reforzar los conceptos 📖"

# 2. Variables y Colección (Cajas en memoria)
estudiantes = ["Ana", "Carlos", "Sofía"]
calificaciones = [9.5, 5.8, 8.2]

# 3. Bucle de Procesamiento (Cinta Transportadora)
for i in range(len(estudiantes)):
    resultado = evaluar_estudiante(estudiantes[i], calificaciones[i])
    print(resultado)
```

Análisis del Código Fuente

El código define una función pura con type hints, itera una colección de datos mediante un bucle for y delega la toma de decisiones al condicional interno.

4. Buenas Prácticas, Gotchas y Depuración

Consejos clave para evitar los errores más comunes al dar tus primeros pasos en Python:

⚠ Gotcha Frecuente (Trampa de Principiante)

Olvidar los dos puntos (:) al final de las estructuras if, for o def, o mezclar espacios y tabulaciones en la indentación.

Patrón Recomendado vs Antipatrón

✗ Antipatrón / Mal Código

```
if nota > 5
print("Aprobado") # Error de sintaxis
```

✓ Patrón Pythonic / Correcto

```
if nota > 5:
    print("Aprobado") # Correcto e indentado
```

🛡 Consejo de Resiliencia en Producción

Configura VS Code para insertar 4 espacios automáticos al presionar la tecla Tab y activa el formateador black o ruff.

5. Resumen Ejecutivo y Conclusiones

Has adquirido el mapa completo del territorio de la programación. Ya conoces los 4 pilares y cómo se coordinan.



Logro Alcanzado en esta Clase

Primer script integrador ejecutado con éxito y comprensión clara del flujo lógico.

Notas del Instructor y Siguientes Pasos

En la próxima sesión profundizaremos en el Almacén de Datos: tipos primitivos, conversión de tipos e interacción con el usuario mediante `input()`.



Mensaje de Agradecimiento

Muchas gracias por tu entusiasmo, disciplina y dedicación al participar en este programa formativo. La programación es un superpoder que transforma vidas cuando se ejerce con constancia y curiosidad. ¡Nos vemos en la próxima sesión para seguir construyendo juntos!

«El código más elegante es aquel que cualquiera puede entender y nadie necesita reescribir.»

6. Fuentes Bibliográficas y Recursos de Estudio

Para profundizar y consolidar los conocimientos adquiridos en esta clase, consulta las siguientes referencias:

Recurso / Fuente	Descripción Temática	Enlace Oficial
Documentación Oficial de Python	Referencia canónica del lenguaje y librería estándar	docs.python.org/3/
PEP 8 - Style Guide for Python	Guía oficial de estilo, formato e indentación	peps.python.org
Real Python Tutorials	Artículos técnicos y patrones de desarrollo moderno	realpython.com
Python Type Checking (PEP 484)	Anotaciones de tipo y análisis estático en Python	docs.python.org/typing
Suite Open Source wisrovi	Paquetes Python para orquestación y alto rendimiento	github.com/wisrovi

Canales de Consulta y Comunidad

Si encuentras dificultades al replicar el código o tienes dudas sobre la arquitectura de los ejercicios, no dudes en abrir un Issue en el repositorio oficial del curso en GitHub o participar activamente en nuestras sesiones grupales de mentoría.



Desafío de Autoestudio Recomendado

Modifica el script de la página 6 para que evalúe a 5 alumnos y clasifique notas con honores (mayores a 9.0).